

FAETERJ-Rio
Organização de Computadores – 1ORG
Memória – parte 2

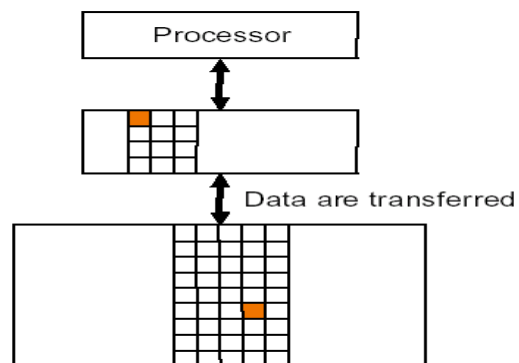
Prof. Paulo Massillon

Memória – Organização

Tamanho
Endereço
Célula - Unidade de Endereçamento
Bloco
Largura de Transferência
Palavra
Conteúdo



Memória – Transferência



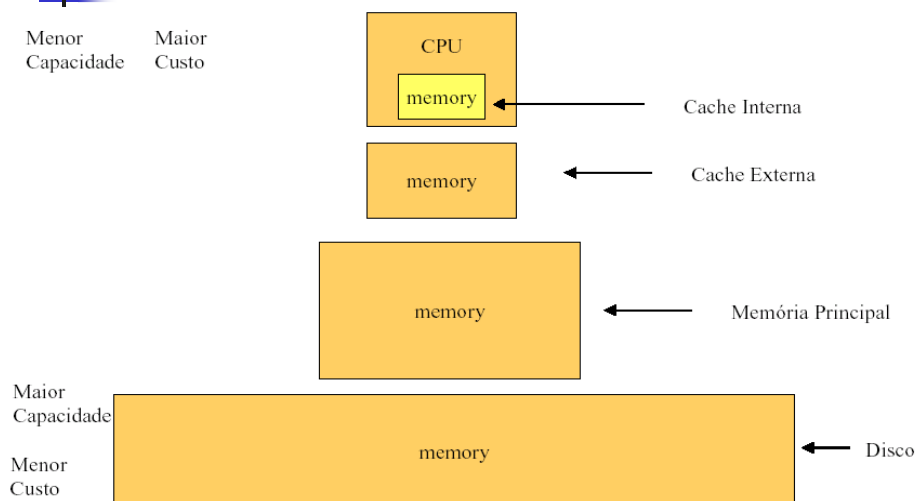
3



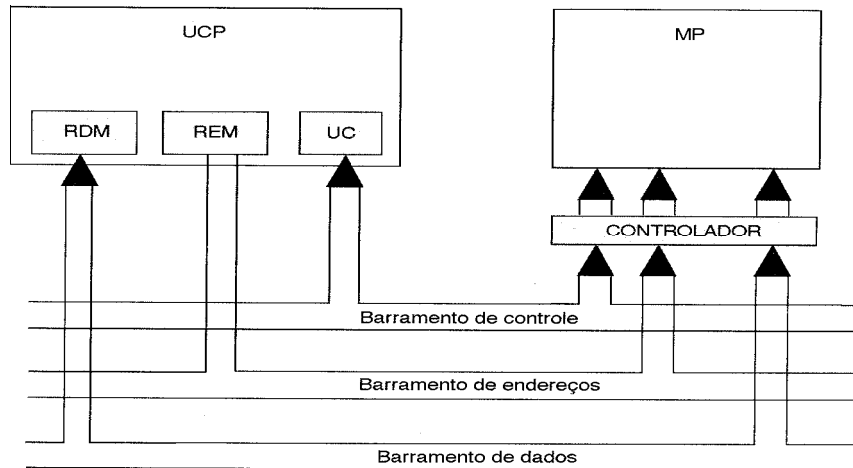
Memória – Hierarquia

Menor
Capacidade

Maior
Custo



Memória Principal – Movimentação



Fonte: Mário Monteiro, Arquitetura de Computadores, pág. 126

5

Memória Principal – Leitura

- REM $\leftarrow$ de outro registrador da UCP
 - Endereço colocado no barramento de endereços
- Sinal de leitura é colocado no barramento de controle
 - Decodificação do endereço e localização da célula
- RDM $\leftarrow$ MP(REM) pelo barramento de dados
- Outro registrador da UCP $\leftarrow$ RDM

Fonte: Mário Monteiro, Arquitetura de Computadores, pág. 127

6



Memória Principal – Gravação

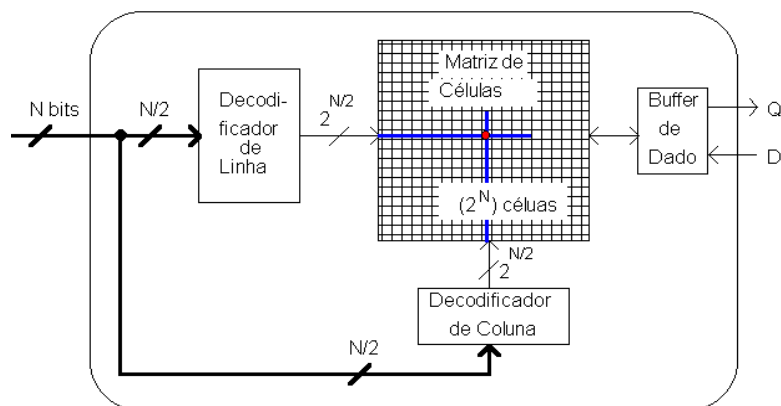
- $REM \leftarrow$ de outro registrador da UCP
 - Endereço colocado no barramento de endereços
- $RDM \leftarrow$ conteúdo de outro registrador
- Sinal de escrita é colocado no barramento de controle
- $MP(REM) \leftarrow RDM$ pelo barramento de dados

Fonte: Mário Monteiro, Arquitetura de Computadores, pág. 128

7



Acesso à Memória Principal



8



Memória Cache

- Diferença de velocidade entre UCP e Memória Principal
 - Wait-state
 - Relógio – gerador de pulsos
 - Frequência – pulsos por segundo
 - Ciclo de relógio ou de máquina – entre pulsos
 - Sub-ciclos

9



Memória – Princípio da Localidade

- Programas repetem trechos de código e acessam repetidamente dados próximos
 - localidade temporal: posições de memória, uma vez acessadas, tendem a ser acessadas novamente no futuro próximo (loops)
 - localidade espacial: endereços em próximos acessos tendem a ser próximos de endereços de acessos anteriores (instruções)

10



Memória – Cache - Níveis

L1 (level 1) – no interior da pastilha do processador

L2 (level 2) – na placa-mãe ou, em casos, na pastilha onde está o processador (mas não no processador)

L3 (level 3) – externa ao processador

11



Memória – Cache - Conceitos

"Hit"

"Miss"

Tempo de "Hit"

Penalidade de "Miss"

"Hit" ratio (h)

"Miss" ratio (1 – h)

12



Memória – Impacto de um “Hit”

- $T_{ma} = T_h + (1 - h) T_m$

- Exemplo:

- $T_h = 2 \text{ ns}$
- $T_m = 40 \text{ ns}$
- $h = 0,95$

então $T_{ma} = 4 \text{ ns}$

13



Memória Cache – Projeto

- Definição dos tamanhos da L1, L2 e L3
- Função de mapeamento dos dados MP/cache
- Algoritmo de substituição de dados na cache
- Política de escrita na cache

14



Memória Cache – Definição de tamanho

- Menor possível (\$)
- Fatores a serem considerados:
 - Tamanho da Memória Principal
 - Relação acertos(hits)/falhas(misses)
 - Tempo de acesso da Memória Principal
 - Custo médio por bit, da MP, da L1 e da L2
 - Tempo de acesso da L1 e da L2
 - Natureza do programa em execução (Localidade)

15



Memória Cache – Definição de tamanho

Processadores	Fabricante	Tamanho L1
486	Intel	8 KB - única
C6	Cyrix	64 KB - dividida
K5	AMD	24 KB - dividida
K7	AMD	128 KB - dividida
Pentium	Intel	16 KB - dividida
Pentium MMX	Intel	16 KB - dividida
Pentium PRO	Intel	64 KB - dividida
Power PC 601	Motorola / IBM	32 KB
Pentium III	Intel	32 KB - dividida

Fonte: Mário Monteiro, Arquitetura de Computadores, pág. 147

16



Parâmetros de Cache L1 – i7

Table 2-9. Cache Parameters

Level	Capacity	Associativity (ways)	Line Size (bytes)	Write Update Policy	Inclusive
L1 Data	32 KB	8	64	Writeback	-
Instruction	32 KB	8	N/A	N/A	-
L2 (Unified)	256 KB	8	64	Writeback	No
Third Level (LLC)	Varies, query CUID leaf 4	Varies with cache size	64	Writeback	Yes

17



Parâmetros de Cache – i7

Characteristic	ARM Cortex-A8	Intel Nehalem
L1 cache organization	Split instruction and data caches	Split instruction and data caches
L1 cache size	32 KiB each for instructions/data	32 KiB each for instructions/data per core
L1 cache associativity	4-way (I), 4-way (D) set associative	4-way (I), 8-way (D) set associative
L1 replacement	Random	Approximated LRU
L1 block size	64 bytes	64 bytes
L1 write policy	Write-back, Write-allocate(?)	Write-back, No-write-allocate
L1 hit time (load-use)	1 clock cycle	4 clock cycles, pipelined
L2 cache organization	Unified (instruction and data)	Unified (instruction and data) per core
L2 cache size	128 KiB to 1 MiB	256 KiB (0.25 MiB)
L2 cache associativity	8-way set associative	8-way set associative
L2 replacement	Random(?)	Approximated LRU
L2 block size	64 bytes	64 bytes
L2 write policy	Write-back, Write-allocate (?)	Write-back, Write-allocate
L2 hit time	11 clock cycles	10 clock cycles
L3 cache organization	-	Unified (instruction and data)
L3 cache size	-	8 MiB, shared
L3 cache associativity	-	16-way set associative
L3 replacement	-	Approximated LRU
L3 block size	-	64 bytes
L3 write policy	-	Write-back, Write-allocate
L3 hit time	-	35 clock cycles

18



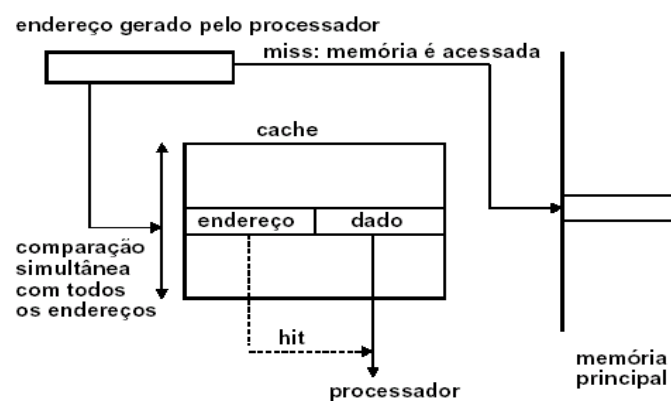
Cache – Funções de Mapeamento

- Completamente Associativo
- Direto
- Associativo por Conjuntos
 - Conjunto-Associativo

19

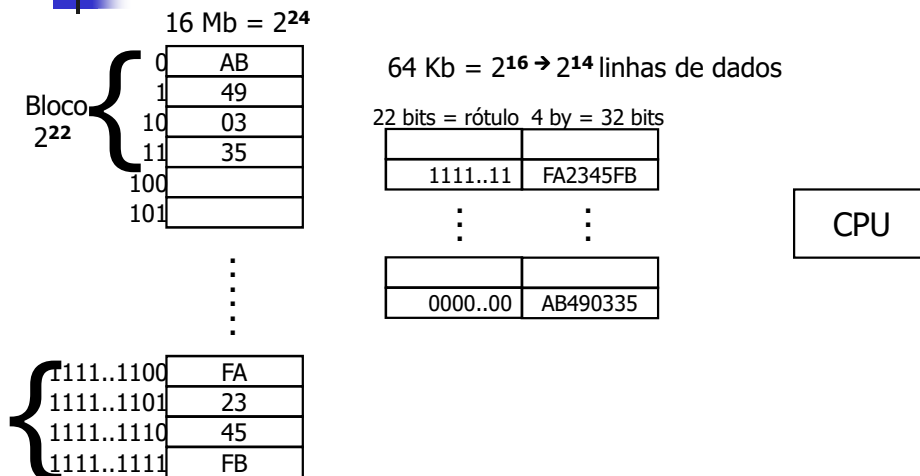


Cache – Mapeamento Completamente Associativo



20

Cache – Mapeamento Completamente Associativo



21

Cache – Mapeamento Completamente Associativo

Vantagem:

máxima flexibilidade no posicionamento de qualquer bloco da memória principal em qualquer linha do cache

Desvantagens:

custo em hardware da comparação simultânea de todos os endereços armazenados no cache

é necessário um algoritmo de substituição (em hardware) para selecionar uma linha do cache como consequência de um "miss", com MC cheia

Utilizado apenas em memórias associativas de pequeno tamanho

22



Exercícios

- Determine, em hexadecimal, o bloco onde estão localizadas as seguintes células, cujos endereços com 32 bits são (bloco de 8 células):
 - **FF8765FA**

 - **AB35867F**

23



Exercícios

- Determine, em hexadecimal, o bloco onde estão localizadas as seguintes células, cujos endereços com 32 bits são (bloco de 8 células):
 - **FF8765FA**
 - Célula em binário: 1111.1111.1000.0111.0110.0101.1111.1010
 - Bloco em binário: 1.1111.1111.0000.1110.1100.1011.1111(010)
 - Bloco em hexadecimal: 1FF0ECBF
 - **AB35867F**
 - Célula em binário: 1010.1011.0011.0101.1000.0110.0111.1111
 - Bloco em binário: 1.0101.0110.0110.1011.0000.1100.1111 (111)
 - Bloco em hexadecimal: 1566B0CF

24



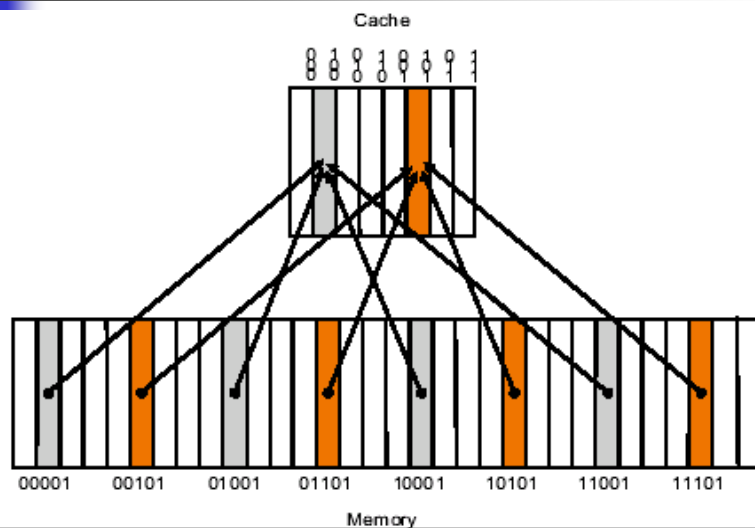
Exercícios

- Determine, em hexadecimal, o bloco onde estão localizadas as seguintes células, cujos endereços com 32 bits são (bloco de 16 células):
 - FF8765FA
 - Bloco em hexadecimal: FF8765F
 - AB35867F
 - Bloco em hexadecimal: AB35867

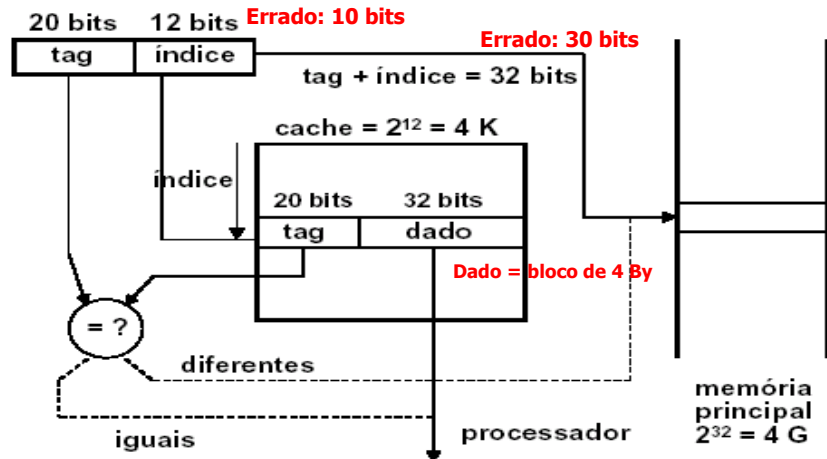
25



Cache – Mapeamento Direto



Cache – Mapeamento Direto



27

Cache – Mapeamento Direto – valores referentes ao slide anterior

16 Mb = 2^{24}

Bloco 2^{22}	0	AB
	1	49
	10	03
	11	35
	100	
	101	
	...	
	1111..1100	FA
	1111..1101	23
	1111..1110	45
	1111..1111	FB

Divisão do endereço de 2^{24} :

- 8 bits – rótulo (TAG)
- 14 bits – linha da cache (índice)
- 2 bits – bloco (não é escrito)

Ex: Bloco_MP = 42 = 000..101010 (2^{22})

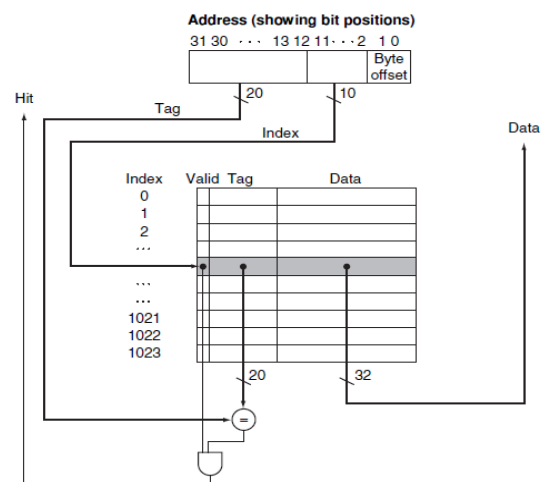
Nº linhas = $64\text{k} / 4 = 16\text{K}$ (2^{14})

Linha_cache = 2^{14} ($24 - 2$ (bloco) - 8 (tag))
= 00 0000 0010 1010

28



29



30



Cache – Mapeamento Direto

Endereço do bloco é dividido em 2 partes:

- parte menos significativa: índice, usado como endereço no cache onde será armazenado o bloco (na cache, bloco = linha)
- parte mais significativa: tag, armazenado no cache junto com o conteúdo do bloco de memória (linha)

Quando acesso é feito, índice é usado para encontrar o bloco no cache

- se tag armazenado na linha do cache é igual ao tag do endereço procurado, então houve "hit"

Endereços com mesmo índice são mapeados sempre para a mesma linha do cache

- no exemplo, são mapeados para a linha de endereço 1 do cache os endereços 1, K+1, 2K+1, 3K+1, etc.

31



Exercício

- Seja um computador com MP de 4GB e MC de 2MBy. Sabendo-se que um bloco da MP tem 16 células, quantos blocos apontarão para o mesmo índice na MC?
- MP $\rightarrow$ 4GB = 2^{32} By
- 2^{32} By em blocos de 16 = 2^4 By $\rightarrow$ 2^{28} blocos, 28 bits para identificar cada bloco.
- MC $\rightarrow$ 2MBy = 2^{21} Bytes $\rightarrow$ 2^{21} bytes / 2^4 Bytes/linha = 2^{17} linhas $\rightarrow$ INDICE tem 17 bits
- Preciso 28 bits para identificar o bloco, TAG = $28 - 17 = 11$ bits
- Daí, 2^{11} blocos apontam para o mesmo índice na MC

32



Cache – Mapeamento Direto

Vantagens

- não há necessidade de algoritmo de substituição
- hardware simples e de baixo custo
- alta velocidade de operação

Desvantagens

- desempenho cai se acessos consecutivos são feitos a diferentes blocos com mesmo índice
- hit ratio inferior ao de caches com mapeamento associativo

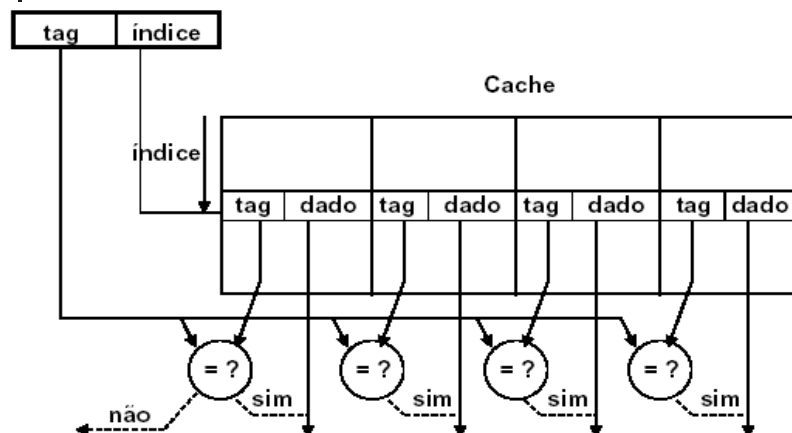
Demonstra-se no entanto que hit ratio aumenta com o aumento do cache, aproximando-se de caches com mapeamento associativo

Tendência atual é de uso de caches grandes

33

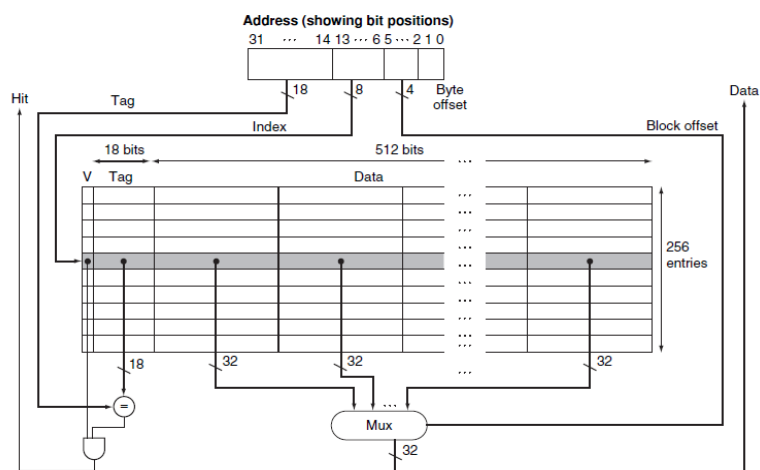


Cache – Mapeamento Associativo por Conjuntos



34

Cache – Mapeamento Associativo por Conjuntos



35

Exercícios

- Usando as informações dos computadores definidos a seguir, calcule:
 - Para Mapeamento Associativo, o número de bits necessários para identificar o bloco;
 - Para Mapeamento Direto, quantidade de bits necessários para compor o TAG e o Índice;
 - Para Mapeamento Conjunto-Associativo, quantidade de bits necessários para compor o TAG e o Índice.

	Comp 1	Comp 2	Comp 3	Comp 4
MP	32 GBy	1 TBy	512 GBy	64 GBy
MC	4 MBy	8 MBy	128 KBy	1 MBy
Células/Bloco	8	8	8	16
Bytes/Célula	1	2	2	1
Linhas/Conjunto	4	8	2	2



Exercícios

- Usando as informações dos computadores definidos a seguir, calcule:
 - Para Mapeamento Associativo, o número de bits necessários para identificar o bloco;
 - Para Mapeamento Direto, quantidade de bits necessários para compor o TAG e o Índice;
 - Para Mapeamento Conjunto-Associativo, quantidade de bits necessários para compor o TAG e o Índice.

	Comp 1	Comp 2	Comp 3	Comp 4
MP	32 GBy	1 TBy	512 GBy	64 GBy
MC	4 MBy	8 MBy	128 KBy	1 MBy
Células/Bloco	8	8	8	16
Bytes/Célula	1	2	2	1
Linhas/Conjunto	4	8	2	2
Associativo (bloco)	32	36	35	32
Direto (TAG/Índice)	13/19	17/19	22/13	16/16
ConjAssoc (TAG/Índice)	15/17	20/16	23/12	17/15



Exercícios

- Memória Principal (COMP 3)**
 - $512 \text{ GBy} = 2^{39} \text{ By}$
 - $2^{39} \text{ By} / 2^1 \text{ By} / \text{célula} = 2^{38} \text{ células}$
 - $2^{38} \text{ células} / 2^3 \text{ células} / \text{bloco} = 2^{35} \text{ blocos} \rightarrow 35 \text{ bits são necessários para identificar um bloco (ASSOCIATIVO)}$
- Memória Cache**
 - $128 \text{ KBy} = 2^{17} \text{ By}$
 - $2^{17} \text{ By} / 2^1 \text{ bytes} / \text{célula} = 2^{16} \text{ células}$
 - $2^{16} \text{ células} / 2^3 \text{ células} / \text{linha} = 2^{13} \text{ linhas}$
- Mapeamento Associativo $\rightarrow 35 \text{ bits}$**
- Mapeamento Direto $\rightarrow \text{ÍNDICE} = 13, \text{TAG} = 35 - 13 = 22$**
- Mapeamento Conjunto Associativo**
 - $2^{13} \text{ linhas} / 2^1 \text{ linhas} / \text{conjunto} = 2^{12} \text{ conjuntos}$
 - ÍNDICE $\rightarrow 12 \text{ BITS}$**
 - TAG $= 35 - 12 = 23$**



Exercícios

- Usando as informações dos computadores definidos a seguir, calcule:
 - Para Mapeamento Associativo, o número de bits necessários para identificar o bloco;
 - Para Mapeamento Direto, quantidade de bits necessários para compor o TAG e o Índice;
 - Para Mapeamento Conjunto-Associativo, quantidade de bits necessários para compor o TAG e o Índice.

	Comp 1	Comp 2	Comp 3	Comp 4
MP	128 GBy	2 TBy	256 GBy	16 GBy
MC	2 MBy	16 MBy	64 KBy	16 MBy
Células/Bloco	8	8	8	16
Bytes/Célula	1	2	2	1
Linhas/Conjunto	4	8	2	2
Associativo (bloco)	34	37	34	30
Direto (TAG/Índice)	16/18	17/20	22/12	10/20
ConjAssoc (TAG/Índice)	18/16	20/17	23/11	11/19



Exercícios

- Memória Principal (COMP 3)**
 - $256 \text{ GBy} = 2^{38} \text{ By}$
 - $2^{38} \text{ By} / 2^1 \text{ By} / \text{célula} = 2^{37} \text{ células}$
 - $2^{37} \text{ células} / 2^3 \text{ células} / \text{bloco} = 2^{34} \text{ blocos} \rightarrow 34 \text{ bits são necessários para identificar um bloco (ASSOCIATIVO)}$
- Memória Cache**
 - $64 \text{ KBy} = 2^{16} \text{ By}$
 - $2^{16} \text{ By} / 2^1 \text{ bytes} / \text{célula} = 2^{15} \text{ células}$
 - $2^{15} \text{ células} / 2^3 \text{ células} / \text{linha} = 2^{12} \text{ linhas}$
- Mapeamento Associativo $\rightarrow 34 \text{ bits}$**
- Mapeamento Direto $\rightarrow \text{ÍNDICE} = 12, \text{TAG} = 34 - 12 = 22$**
- Mapeamento Conjunto Associativo**
 - $2^{12} \text{ linhas} / 2^1 \text{ linhas} / \text{conjunto} = 2^{11} \text{ conjuntos}$
 - ÍNDICE $\rightarrow 11 \text{ BITS}$**
 - TAG $= 34 - 11 = 23$**



Exercícios

- Seja um computador cuja memória cache, em mapeamento associativo por conjuntos, com 8 linhas por conjunto, é acessada através de um índice de 14 bits, e usa para comparar a pertinência do bloco um tag de 16 bits.
- Sabe-se adicionalmente que as células desse computador tem 2 bytes, e os blocos são compostos por 8 células.
- Informe o tamanho, em bytes, da memória principal e da memória cache.



Exercícios

- Se o índice tem 14 bits, significa que temos 2^{14} conjuntos.
- Cada conjunto tem 8 linhas, então 2^{14} conjuntos * 2^3 linhas / conjunto = 2^{17} linhas.
- Sendo cada linha (bloco) composta de 8 células, então temos 2^{17} linhas * 2^3 células / linha = 2^{20} células.
- Uma vez que cada célula tem 2 bytes, a MEMÓRIA CACHE é composta de 2^{20} células * 2^1 bytes / célula = 2^{21} bytes → 2 MBy.
- A soma dos bits do Índice com os bits do TAG nos dá a quantidade de bits necessária para identificar um bloco, portanto, $14 + 16 = 30$ bits identifica um bloco, existindo portanto 2^{30} blocos.
- Se cada bloco tem 8 células, então temos 2^{30} blocos * 2^3 células / bloco = 2^{33} células, cada uma de 2 bytes, e portanto a MEMÓRIA PRINCIPAL tem 2^{33} células * 2^1 bytes / célula = 2^{34} bytes → 16 GB



Exercícios

- Usando as informações dos computadores definidos a seguir, calcule, para mapeamento conjunto-associativo:
 - Tamanho em bytes da Memória Principal
 - Tamanho em bytes da Memória Cache

	Comp 1	Comp 2	Comp 3	Comp 4
Índice	17 bits	16 bits	12 bits	15 bits
TAG	15 bits	20 bits	23 bits	17 bits
Células/Bloco	8	8	8	16
Bytes/Célula	1	2	2	1
Linhas/Conjunto	4	8	2	2
Memória Principal				
Memória Cache				



Cache – Políticas de escrita

- Write-through
 - cada escrita no cache é repetida imediatamente na memória principal
 - escrita adicional na memória principal reduz tempo médio de acesso ao cache
 - estatisticamente apenas 5% a 34% dos acessos à memória são escritas



Cache – Políticas de escrita

- **Write-back**
 - linha da cache só é escrita de volta no bloco da memória principal quando precisa ser substituída e foi alterada
 - estratégia mais simples: escrita é feita mesmo que linha não tenha sido alterada
 - estratégia alternativa: só escrever de volta se linha foi alterada
 - exige-se um bit de marcação para indicar modificações na palavra

45



Cache – Políticas de escrita

- **Write-once**
 - palavra da memória principal é atualizada simultaneamente à alteração do valor da palavra na memória cache
 - estratégia usada com múltiplos processadores
 - Outros componentes que compartilham o barramento são alertados

46



Cache – Algoritmo de Substituição

- Quando ocorre um miss, um novo bloco precisa ser trazido da memória principal para a cache, que pode estar completamente cheia
 - cache com mapeamento direto não precisa escolher qual linha da cache será substituída
 - cache completamente associativa: pode-se escolher qualquer um das linhas
 - cache conjunto-associativa: deve-se escolher uma linha dentro de um conjunto fixado pelo índice
 - algoritmo de substituição precisa ser implementado em hardware
 - substituição randômica
 - FIRST-IN FIRST-OUT
 - LRU – Least Recently Used

47



FIM
Memória – parte 2

Prof. Paulo Massillon